COMBINE FOUNDTION

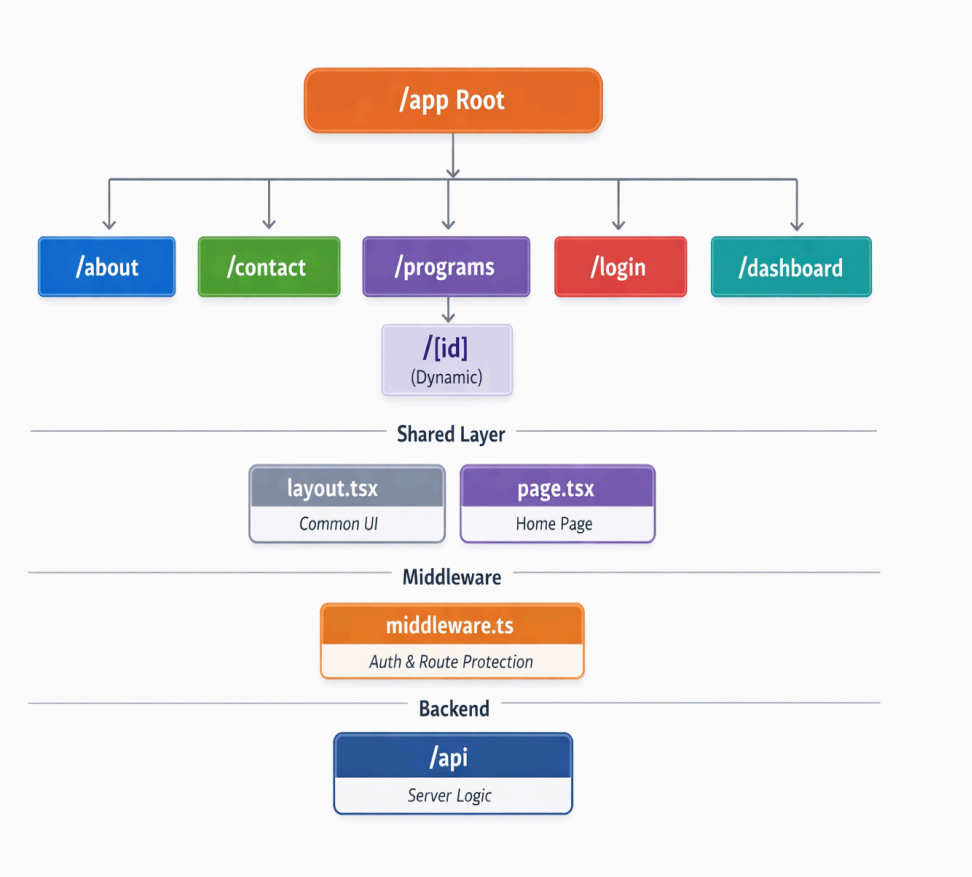
Tech Architecture Document – Next.js App Router

Overview

This document defines the architecture of the project using Next.js App Router. It ensures all developers follow a consistent structure, coding pattern, and decision-making approach.

Project Structure

Root Structure
/app
/about
/contact
/programs
/[id]
/login
/dashboard
/api
layout.tsx
page.tsx
middleware.ts



1. Routing (App Router)

Static Routes

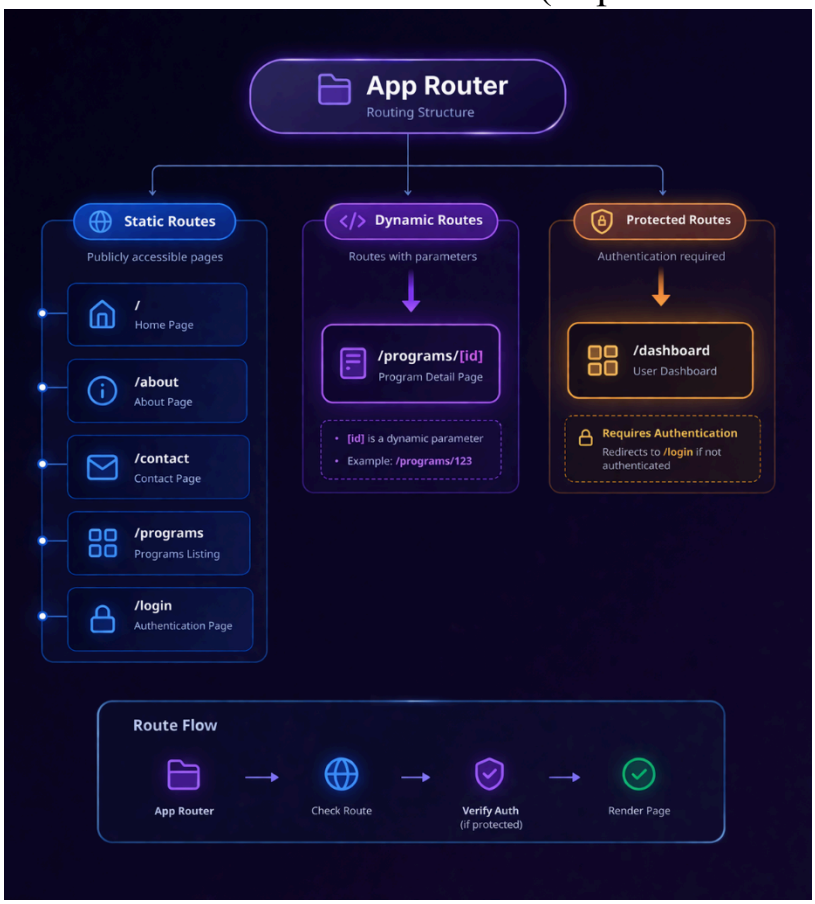
- / → Home Page
- /about → About Page
- /contact → Contact Page
- /programs → Programs Listing
- /login → Authentication Page

Dynamic Routes

- /programs/[id] → Program Detail Page

Protected Routes

- /dashboard → User Dashboard (requires authentication)



Layout System

Global Layout

- File: app/layout.tsx
- Contains:
    - Navbar
    - Footer
    - Global styles

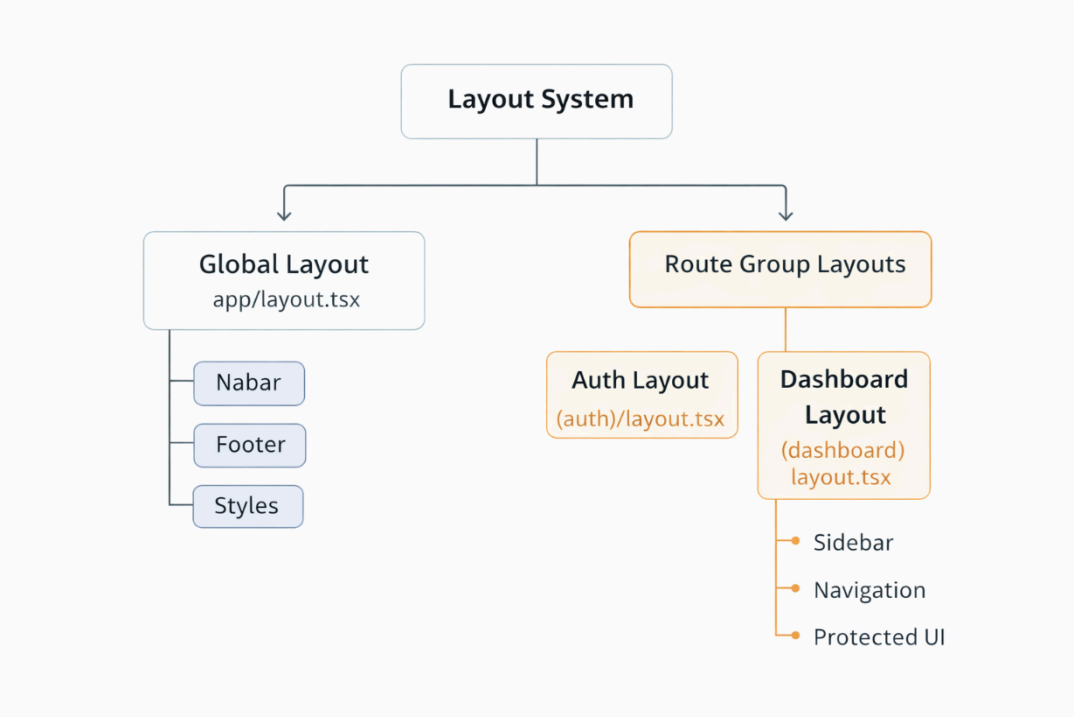
Route Group Layouts

Auth Layout

- Path: app/(auth)/layout.tsx
- Used for login/register pages

Dashboard Layout

- Path: app/(dashboard)/layout.tsx
- Contains sidebar, navigation, protected UI



---

2. Server vs Client Components

Server Components (Default)

Used for:

- Data fetching
- Static content
- SEO pages

Advantages:

- Better performance
- Reduced client-side JavaScript
- Secure data handling

Client Components

Declared using:

"use client"

Used for:

- Forms (login, contact)
- Interactive UI (modals, dropdowns)
- State management (useState, useEffect)

Rule

- Always use Server Components by default
- Switch to Client Components only when interaction is required

---

3. API Routes

Location

app/api/

Endpoints

Auth

- POST /api/auth/login
- POST /api/auth/logout

Programs

- GET /api/programs → Get all programs
- GET /api/programs/[id] → Get single program

Contact

- POST /api/contact → Submit contact form

Responsibilities

- Handle business logic
- Communicate with database (Firebase)
- Process user input securely

---

4. Middleware

File
middleware.ts
Responsibilities

- Protect private routes
- Handle redirects
- Manage authentication checks

Logic

- If user is NOT authenticated:
  → Redirect to /login
- If user IS authenticated:
  → Allow access to /dashboard
- Optional:
  - Prevent logged-in users from accessing /login

Authentication Flow

1. User visits /login
2. Credentials sent to /api/auth/login
3. Server validates user (Firebase)
4. Session/token stored
5. Middleware allows access to protected routes

Data Flow

Server Component Flow

1. Request page

2. Fetch data on server
3. Render HTML
4. Send to client

Client Component Flow

1. User interacts (form/button)
2. API call made
3. Response handled in UI

Firebase Setup

Overview

This document defines how Firebase is configured and used across the project. It ensures consistent implementation of authentication, database, storage, and hosting decisions.

---

1. Authentication

Supported Providers

- Email / Password (Primary)
- Google Sign-In (Optional / Recommended)

Configuration

- Enable providers in Firebase Console → Authentication → Sign-in method
- Use Firebase Auth SDK on client side for login/signup

Usage

- Login handled via API route: /api/auth/login
- Logout handled via API route: /api/auth/logout

Security Rules

- Only authenticated users can access protected resources
- Use ID tokens to verify user identity on server

---

2. Firestore (Database)

Environment Separation

Development Project

- Used for testing and development
- Contains dummy/sample data

Production Project

- Used for live users
- Strict security rules applied

Configuration Strategy

- Use .env.local for dev Firebase config
- Use .env.production for production config

Collections Structure (Example)

users/
programs/
applications/
messages/

Rules

- Restrict read/write access based on authentication
- Validate user ownership of data

---

3. Storage (File Management)

Purpose

- Store uploaded files such as:
    - User documents
    - Images
    - Program-related files

Folder Structure

/uploads
  /users/{userId}/
  /programs/{programId}/
  /general/

Rules

- Users can only upload/read their own files
- Admin-level access for program-related uploads

---

4. Hosting Decision

Selected: Vercel

Reason for Choosing Vercel

- Native support for Next.js (App Router)
- Automatic deployments from Git
- Built-in serverless functions (API routes)
- Better performance for SSR (Server-Side Rendering)

Why NOT Firebase Hosting

- Limited support for advanced Next.js features
- More complex setup for SSR

Deployment Workflow

Development

- Run locally using Firebase dev config
- Use test database and storage

Production

- Deploy via Vercel
- Use production Firebase project
- Environment variables configured in Vercel dashboard

Security Best Practices

- Never expose private Firebase keys
- Use environment variables for all configs
- Validate all requests on server (API routes)
- Apply strict Firestore and Storage rules

Folder Structure Convention

Overview

This document defines the standard folder structure for the project. It ensures consistency, scalability, and maintainability across the codebase.

Root-Level Structure

/components
/lib
/app

/public
Each folder has a specific responsibility and must be used accordingly.

---

1. components/ (Reusable UI Components)

Purpose

- Store all reusable UI components
- Keep UI logic separate from routing and business logic

Examples

```
/components
  /ui
    Button.tsx
    Input.tsx
  /layout
    Navbar.tsx
    Footer.tsx
  /shared
    Card.tsx
```

Guidelines

- Components must be reusable and generic
- Avoid business logic inside components
- Use Client Components only when required ("use client")

---

2. lib/ (Utilities & Firebase Setup)

Purpose

- Central place for all utility functions and external integrations

Examples

/lib
firebase.ts
auth.ts
helpers.ts
api.ts

Responsibilities

- Firebase initialization
- Authentication helpers
- Common utility functions
- API helper methods

Guidelines

- No UI code in this folder
- Keep functions pure and reusable

---

3. app/ (Next.js App Router)

Purpose

- Contains all routes, pages, layouts, and API endpoints

Structure Example

```
/app
  /about
    page.tsx
  /programs
    page.tsx
    /[id]
      page.tsx
  /dashboard
    page.tsx
  /api
    /auth
    /programs
```

Responsibilities

- Page routing
- Layout management
- Server components
- API routes

Guidelines

- Follow file-based routing strictly
- Keep pages clean; move logic to lib/

---

4. public/ (Static Assets)

Purpose

- Store static files accessible directly via URL

Examples

/public
  /images
  /icons
  favicon.ico

Usage

- Access via: /images/image-name.png

Guidelines

- Only static files (no dynamic content)
- Optimize images for performance

---

7. Naming Conventions

- Use lowercase for folders
- Use PascalCase for React components (e.g., Button.tsx)
- Use clear and descriptive names

---

8. Best Practices

- Keep folder responsibilities strictly separated
- Avoid deeply nested structures unless necessary
- Reuse components instead of duplicating code
- Centralize logic inside lib/